INTRODUCTION

1.INTRODUCTION

Fraud is a wrongful or criminal deception aimed to bring financial or personal gain [1]. In avoiding loss from fraud, two mechanisms can be used: fraud prevention and fraud detection. Fraud prevention is a proactive method, where it stops fraud from happening in the first place. On the other hand, fraud detection is needed when a fraudulent transaction is attempted by a fraudster. Credit card fraud is concerned with the illegal use of credit card information for purchases. Credit card transactions can be accomplished either physically or digitally [2]. In physical transactions, the credit card is involved during the transactions. In digital transactions, this can happen over the telephone or the internet. Cardholders typically provide the card number, expiry date, and card verification number through telephone or website. With the rise of e-commerce in the past decade, the use of credit cards has increased dramatically [3]. The number of credit card transactions in 2011 in Malaysia were at about 320 million, and increased in 2015 to about 360 million. Along with the rise of credit card usage, the number of fraud cases have been constantly increased. While numerous authorization techniques have been in place, credit card fraud cases have not hindered effectively. Fraudsters favour the internet as their identity and location are hidden. The rise in credit card fraud has a big impact on the financial industry. The global credit card fraud in 2015 reached to a staggering USD $21.84 billion [4]. Loss from credit card fraud affects the merchants, where they bear all costs, including card issuer fees, charges, and administrative charges [5]. Since the merchants need to bear the loss, some goods are priced higher, or discounts and incentives are reduced. Therefore, it is imperative to reduce the loss, and an effective fraud detection system to reduce or eliminate fraud cases is important. There have been various studies on credit card fraud detection. Machine learning and related methods are most commonly used, which include artificial neural networks, rule-induction techniques, decision trees, logistic regression, and support vector machines [1]. These methods are used either standalone or by combining several methods together to form hybrid models. IEEE In this paper, a total of twelve machine learning algorithms are used for detecting credit card fraud. The algorithms range from standard neural networks to deep learning models. They are evaluated using both benchmark and realworld credit card data sets. In addition, the AdaBoost and majority

voting methods are applied for forming hybrid models. To further evaluate the robustness and reliability of the models, noise is added to the real-world data set. The key contribution of this paper is the evaluation of a variety of machine learning models with a real-world credit card data set for fraud detection. While other researchers have used various methods on publicly available data sets, the data set used in this paper are extracted from actual credit card transaction information over three months.

1.1ABOUT ORGANIZATION

HISTORY OF HMIES SOFTWARE SOLUTIONS

Foundation:

Founded in 1992, company has grown phenomenally to becoming the most sought for by majority of the student and institutional community in Coastal Andhra Pradesh. With over 100 staff in training and around 200 staff in software development, the company is all set to jet accelerate its growth in the time around with introduction of more technologies and methodologies that make it contemporary**.**

About Company:

HMIES was incorporated in 2000 with a vision to be the recognized leader in providing reliable business solutions world – wide for superior business results in the critical areas (e – Business, Business Intelligence, e – Commerce, Mobile Commerce Solutions, Strategic Business Consulting, Collaborative Business Solutions including Enterprise Information System and Enterprise Application Integration Solutions) high end training through computing and creative excellence. Committed to delivering the best solution for each client, we fuse technology with business strategy to address our clients' current needs and position them for future opportunities. Our expertise and experience, teamed with our extensive knowledge in business application development, allows HMIES to deliver the synergy of creativity and user friendly systems on all platforms to our clients.

The objective is to provide unique, cost-effective and powerful solutions in all domains. The objective has been fulfilled to large extent and today the company is poised for major developments in the high-end technological domains. The technical team constitutes of high caliber IT professionals with proven skill sets in the industry.

HMIES is an excellent professional product Manifacturing Company in the Andhra Pradesh, the most widely used textbook in coaching, and the largest number of

certified coaches globally. To this we have now added the first STPI accredited curriculum on coaching Relationship Systems (teams and partnerships), the most extensive business development program, and the most active coaching community. Discover how co-activity can serve.

HMIES strengths are providing Business to Enterprise as well as BUSINESS _ Integrated Solutions; e-Commerce solutions and the design, development and deployment of enterprise wide applications. We provide stakeholder analysis tools and help our client achieve their organizational objectives by analyzing their current and future critical business challenges. The results of this analysis in turn empowers organizations to formulate and implement global business strategies giving them the cutting edge solutions needed to be a business leader in every sense.

Additional Facilities:

1. To provide placement assistance to 25,000 every year through campus recruitment drives mega job fair and corporate recruitment services.
2. To develop software solutions on many platforms to atleast 50 corporate and assist in their business process.
3. To provide data management services to 25 organizations on contractual basis.

WHO'S WHO:

- Dinesh hirawat — CEO
- N.Akhil Nandina — General manager of operations
- R.Srinivas Rao — General manager of business
- K. Rohith — HR Manager

1.2ABOUT PROJECT

To develop a robust and accurate credit card fraud detection system using a combination of machine learning models, particularly leveraging AdaBoost and majority voting techniques.

Key Highlights:

1. Motivation:

Credit card fraud is a significant issue causing billions in losses.

There's a need for more robust systems capable of detecting fraudulent transactions in real-time using advanced machine learning methods.

2. Techniques Used:

Ada Boost: An ensemble technique that boosts the performance of weak learners.

Majority Voting: Combines predictions from multiple models and selects the output with the most votes.

Hybrid Models: Integration of multiple machine learning models including Decision Trees, Naive Bayes, Random Forests, and Neural Networks.

3. Dataset:

Publicly available datasets.

Real-world transaction data from a financial institution, with added synthetic noise (10%-30%) to test robustness.

4. Technologies Used:

Frontend: HTML, JavaScript

Backend: Java (JSP/Servlets)

Database: MySQL

Server: Apache Tomcat

IDE: Net Beans

1.3 ABOUT FRONT END

What is Front-End Development?

Front-end development is the part of web development that focuses on the **user interface (UI)** — what people **see and interact with** in their browser.

It's all about building the **look, feel, and behavior** of a website or app.

Main Building Blocks of Front-End

1. **HTML – Structure**
   - Defines the content (text, images, buttons).
   - Like the **skeleton** of a webpage.
2. **CSS – Style**
   - Styles the content (colors, fonts, layouts).
   - Like the **skin and clothes** of the webpage.
3. **JavaScript – Behavior**
   - Makes the site interactive (sliders, pop-ups, animations, form validations).
   - Like the **brain** of the page that responds to user actions.

Front-End Workflow

When you visit a website, here's what happens:

- The browser loads the **HTML** to show the structure.
- Then it applies **CSS** for design.
- Finally, **JavaScript** runs to add interactivity.

What Does a Front-End Developer Do?

A front-end developer:

- Builds web pages and apps using HTML, CSS, and JavaScript.
- Ensures the website looks good on **all devices** (responsive design).
- Optimizes websites for **performance and speed**.
- Works with **designers** and **back-end developers**.

Popular Front-End Tools & Frameworks

- **CSS Frameworks**: Bootstrap, Tailwind CSS
- **JavaScript Libraries**: React, Vue, jQuery
- **Package Managers**: npm, yarn
- **Build Tools**: Webpack, Vite
- **Version Control**: Git & GitHub
-

Why Learn Front-End?

- It’s **visual and creative**
- You get **instant results** in the browser
- High demand in the job market
- Fun and rewarding to build things people can use

Would you like to start with a simple front-end project? Or get a roadmap to becoming a front-end developer?

What is HTML?

HTML stands for **HyperText Markup Language**.
It's the **standard language** used to **create and structure webpages**.

Key Points:

- HTML is **not** a programming language — it’s a **markup language**.
- It uses **tags** (like <p>, <h1>, <a>, etc.) to tell the browser how to display content.
- HTML builds the **skeleton** of a webpage — like text, images, links, buttons, and forms.

What is CSS?

CSS stands for **Cascading Style Sheets**.
It is used to **style and layout HTML elements**.

Key Points:

- CSS controls **colors, fonts, margins, layouts, and more**.
- It separates **design** from **content**.
- Can be written:
  - **Inline** (inside HTML tags)
  - **Internal** (inside <style> in the HTML <head>)
  - **External** (linked CSS file like style.css)

What Can You Do with HTML & CSS?

Create websites
Design web layouts
Build forms and interactive pages
Style buttons, animations, and hover effects
Make responsive sites for mobile & desktop

Want to try building your **first mini website**? Or explore a specific concept like **buttons**, **forms**, or **layouts**? Let me know and I’ll guide you step by step!

1.4ABOUT UML

UML is used to visualize the design of the credit card fraud detection system using AdaBoost and Majority Voting. It provides different perspectives (views) of the system through diagrams, each focusing on a specific aspect.

The Five Views of UML:

1. User Model View – Represents the system from the user's perspective (use cases).

2. Structural Model View – Focuses on the data and static structure (class diagrams).

3. Behavioral Model View – Shows dynamic behavior (interactions and state changes).

4. Implementation Model View – Reflects structural and behavioral design elements to be implemented.

5. Environmental Model View – Depicts system configurations and deployment.

USE CASE DIAGRAM:

A use case diagram at its simplest is a representation of a user's interaction with the system and depicting the specifications of a use case. A use case diagram can portray the different types of users of a system and the various ways that they interact with the system. This type of diagram is typically used in conjunction with the textual use case and will often be accompanied by other types of diagrams as well.

CLASS DIAGRAM:

The class diagram is the main building block of object oriented modeling. It is used both for general conceptual modeling of the systematic of the application, and for detailed modeling translating the models into programming code. Class diagrams can also be used for data modeling. The classes in a class diagram represent both the main objects, interactions in the application and the classes to be programmed. A class with three sections, in the diagram, classes is represented with boxes which contain three parts:

The upper part holds the name of the class

The middle part contains the attributes of the class

The bottom part gives the methods or operations the class can take or undertake.

SEQUENCE DIAGRAM:

A sequence diagram is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. A sequence diagram shows object interactions arranged in time sequence. It depicts the objects and classes involved in the scenario and the sequence of messages exchanged between the objects needed to carry out the functionality of the scenario. Sequence diagrams are typically associated with use case realizations in the Logical View of the system under development. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

ACTIVITY DIAGRAM:

An activity diagram is a type of UML (Unified Modeling Language) diagram that visually represents the flow of control or data within a system or process. It is especially useful for modeling the dynamic aspects of a system, such as workflows, operations, or business processes.

PROBLEM STATEMENT

2.PROBLEM STATEMENT

Credit card fraud has become a major issue for financial institutions and consumers worldwide, leading to significant financial losses annually. Despite the growing threat, research on real-world credit card fraud detection remains limited due to confidentiality concerns surrounding sensitive financial data. The primary challenge lies in accurately detecting fraudulent transactions within large volumes of data, which are often highly imbalanced and noisy.

This project aims to develop an effective credit card fraud detection system using a combination of machine learning algorithms. Initially, standard models are employed for baseline performance evaluation. The project then introduces hybrid approaches, specifically using AdaBoost and majority voting techniques, to enhance detection accuracy and robustness. Publicly available datasets and real-world transaction data from financial institutions are utilized to validate the models. Furthermore, noise is deliberately added to assess the resilience of the proposed techniques. The ultimate goal is to achieve a highly accurate and stable system capable of identifying fraudulent transactions even in the presence of data irregularities.

LITERATURE SURVEY

3.LITERATURE SURVEY

Credit card fraud detection is a critical task in financial systems, aiming to distinguish between legitimate and fraudulent transactions. Due to the highly imbalanced and dynamic nature of fraud data, traditional classifiers often struggle to maintain high accuracy and low false positives. Ensemble methods, especially AdaBoost (Adaptive Boosting), combined with majority voting, have been increasingly employed to address these issues.

1. Ensemble Learning and AdaBoost

Freund and Schapire (1997) introduced AdaBoost, a boosting algorithm that combines multiple weak classifiers to form a strong classifier. AdaBoost adjusts the weights of incorrectly classified instances, thus improving the model iteratively.

Opitz and Maclin (1999) emphasized that ensemble methods like boosting significantly improve classification performance, especially for complex datasets such as those in fraud detection.

2. Credit Card Fraud Detection Using AdaBoost

Bhattacharyya et al. (2011) conducted a comparative study of different machine learning algorithms for credit card fraud detection and found that ensemble methods, especially boosting, outperform single classifiers.

Dal Pozzolo et al. (2015) tackled imbalanced fraud datasets and proposed sampling techniques in conjunction with AdaBoost to improve performance.

Carcillo et al. (2018) highlighted that boosting classifiers such as AdaBoost are effective when combined with anomaly detection and data resampling methods.

3. Majority Voting in Ensemble Learning

Kuncheva (2004) described majority voting as a decision fusion technique where the class receiving the highest votes from individual classifiers is selected. This method improves robustness and reduces overfitting.

Verma and Srivastava (2018) evaluated majority voting with multiple ensemble methods for fraud detection, showing that combining AdaBoost with voting can enhance detection rates.

4. Hybrid Models:

AdaBoost with Majority VotingKou et al. (2014) implemented a hybrid model using AdaBoost and majority voting for fraud detection, which resulted in higher detection accuracy and lower false positives**.**

Sahin and Duman (2020) proposed an ensemble model integrating multiple classifiers (AdaBoost, Random Forest, etc.) with majority voting. Their study showed that such combinations yield superior performance in real-time fraud detection systems**.**

Chouhan et al. (2021) investigated the impact of ensemble voting techniques in credit card fraud detection and concluded that majority voting enhances the stability of AdaBoost when used on noisy and imbalanced datasets.

REQUIREMENT ANALYSIS DOCUMENT

4.1 PURPOSE AND SCOPE OF THE SYSTEM

PURPOSE:

The purpose of the credit card fraud detection system is to accurately identify and prevent fraudulent credit card transactions in real-time. By using the AdaBoost ensemble learning algorithm with majority voting, the system enhances the prediction accuracy by combining multiple weak classifiers to form a strong decision-making model. The primary goal is to minimize financial loss, maintain customer trust, and reduce false positives and negatives in fraud detection.

SCOPE:

1. Data Input: The system will take transaction-related data (e.g., amount, time, location, user behaviour patterns).

2. Preprocessing: Includes data cleaning, normalization, and feature extraction.

3. Model Training: Uses AdaBoost to train multiple weak classifiers (e.g., decision stumps) and aggregates them using majority voting for final decision-making.

4. Fraud Detection: Classifies each transaction as either 'fraudulent' or 'legitimate'.

5. Evaluation Metrics: Uses metrics like accuracy, precision, recall, F1-score, and ROC-AUC to assess performance.

6. Deployment: Can be integrated into online banking platforms or merchant payment systems for real-time fraud detection.

4.2EXISTING SYSTEM

Three methods to detect fraud are presented. Firstly, clustering model is used to classify the legal and fraudulent transaction using data clusterisation of regions of parameter value. Secondly, Gaussian mixture model is used to model the probability density of credit card user's past behaviour so that the probability of current behaviour can be calculated to detect any abnormalities from the past behaviour. Lastly, Bayesian networks are used to describe the statistics of a particular user and the statistics of different fraud scenarios. The main task is to explore different views of the same problem and see what can be learned from the application of each different technique.

A credit card fraud detection system was proposed in [8], which consisted of a rule-based filter, Dumpster–Shafer adder, transaction history database, and Bayesian learner. The Dempster–Shafer theory combined various evidential information and created an initial belief, which was used to classify a transaction as normal, suspicious, or abnormal. If a transaction was suspicious, the belief was further evaluated using transaction history from Bayesian learning [8]. Simulation results indicated a 98% true positive rate [8]. A modified Fisher Discriminant function was used for credit card fraud detection in [9]. The modification made the traditional functions to become more sensitive to important instances. A weighted average was utilized to calculate variances, which allowed learning of profitable transactions. The results from the modified function confirm it can eventuate more profit [9].

Association rules are utilized for extracting behaviour patterns for credit card fraud cases in [10]. The data set focused on retail companies in Chile. Data samples were defuzzified and processed using the Fuzzy Query 2+ data mining tool [10]. The resulting output reduced excessive number of rules, which simplified the task of fraud analysts [10]. To improve the detection of credit card fraud cases, a solution was proposed in [11]. A data set from a Turkish bank was used. Each transaction was rated as fraudulent or otherwise. The misclassification rates were reduced by using the Genetic Algorithm (GA) and scatter search. The proposed method doubled the performance, as compared with previous results [11]

1.1.1 Disadvantage:

- There is no Majority Voting technique for credit card fraud detection.

4.3. PROPOSED SYSTEM

Total of twelve machine learning algorithms are used for detecting credit card fraud. The algorithms range from standard neural networks to deep learning models. They are evaluated using both benchmark and real world credit card data sets. In addition, the AdaBoost and majority voting methods are applied for forming hybrid models. To further evaluate the robustness and reliability of the models, noise is added to the real-world data set. The key contribution of this paper is the evaluation of a variety of machine learning models with a real-world credit card data set for fraud detection.

In the proposed system, a total of twelve machine learning algorithms are used for detecting credit card fraud. The algorithms range from standard neural networks to deep learning models. They are evaluated using both benchmark and real world credit card data sets. In addition, the AdaBoost and majority voting methods are applied for forming hybrid models. To further evaluate the robustness and reliability of the models, noise is added to the real-world data set.

The key contribution of this paper is the evaluation of a variety of machine learning models with a real-world credit card data set for fraud detection. While other researchers have used various methods on publicly available data sets, the data set used in this paper is extracted from actual credit card transaction information over three months

1.1.2 Advantages:

- The system is very fast due to AdaBoost Technique.
- Effective Majority Voting techniques.

4.4 SOFTWARE AND HARDWARE REQUIREMENTS

HARDWARE REQUIRED:

- System : Pentium IV 2.4 GHz.
- Hard Disk : 100 GB.
- Monitor : 15 VGA Color.
- Mouse : Logitech.
- RAM : 1 GB.

SOFTWARE REQUIRED:

- Operating system : Windows XP/7/10
- Coding Language : Html, JavaScript, Java/J2EE (Jsp Servlet)
- Development Kit : JDK 1.7
- Database : MySQL
- IDE : Netbeans
- Server : Tomcat 7.0

4.5. FUNCTIONAL REQUIREMENTS

1. Data Acquisition and Preprocessing

To build an effective fraud detection system, it is essential to acquire high-quality data. This requirement focuses on sourcing and preparing both publicly available and real-world credit card transaction datasets. The data must be cleaned to handle missing values, normalize or standardize features, and encode categorical variables appropriately. Furthermore, synthetic noise will be injected into the dataset to simulate real-world inconsistencies or anomalies, such as incomplete or manipulated transaction data. This step ensures that the models developed later can handle irregularities in real-time applications and remain resilient under uncertain or tampered data conditions.

2. Model Implementation

This component involves designing and implementing machine learning models for fraud detection. The system will start with standard classification algorithms such as Logistic Regression, Support Vector Machines (SVM), and Random Forests due to their effectiveness in binary classification problems. Once baseline models are evaluated, more advanced hybrid methods will be developed, such as AdaBoost and ensemble learning via majority voting. These techniques combine the predictive power of multiple models to improve performance and reduce variance, particularly useful for highly imbalanced datasets typical in fraud detection scenarios.

3. Model Training and Evaluation

The models must be trained and validated on both clean and noisy datasets to assess their adaptability and generalization capability. The system will use industry-standard evaluation metrics such as accuracy, precision, recall, F1-score, and the Area Under the Receiver Operating Characteristic Curve (ROC-AUC) to measure performance. This requirement ensures that the best-performing model is selected not only based on overall accuracy but also based on its ability to correctly identify rare fraudulent transactions while minimizing false positives, which are critical in financial applications.

4. Fraud Detection

The core functionality of the system is to accurately detect fraudulent transactions from a stream of credit card data. The implemented models should classify each transaction as either genuine or fraudulent with a high level of confidence. The system will be tested under varying conditions including clean and noisy datasets to determine its robustness. It must show strong performance consistency to ensure it can be deployed in real-world environments where data can be imperfect or adversarially manipulated.

5. Visualization and Reporting

To enhance the interpretability of the results, the system will include comprehensive visualizations such as confusion matrices, ROC curves, and precision-recall curves. These visual tools will help identify where the models succeed or fail. Additionally, a detailed experimental report will be generated, summarizing the performance of each model and configuration. The report will include comparative analyses, insights drawn from the results, and recommendations for future improvements, making it useful for both technical and non-technical stakeholders.

4.6.NON-FUNCTIONAL REQUIREMENTS

1. Performance

The system must provide near real-time fraud detection with a response time ideally under one second per transaction during testing. This is crucial for real-world financial applications where immediate actions are required. Moreover, the system should maintain high accuracy rates (preferably above 90%) while keeping false positives at a minimum to prevent unnecessary transaction rejections or customer inconvenience. Efficient model design and resource utilization will contribute to meeting this requirement**.**

2. Scalability

As the volume of financial transactions grows, the fraud detection system must be capable of scaling accordingly. The underlying architecture should allow for processing larger datasets or even real-time streaming data from multiple sources. This can be achieved by designing modular components, using distributed processing frameworks if necessary, and maintaining a separation between training and prediction pipelines. Scalability ensures that the system remains effective and operational as demands increase over time.

3. Robustness

The system must exhibit resilience to changes and imperfections in the data, including noise, outliers, or minor data corruptions. Fraud detection systems often deal with adversarial scenarios where fraudsters may attempt to disguise fraudulent behaviour. Therefore, the models should be tested under such noisy or perturbed conditions to ensure their stability and performance consistency. This robustness also helps in maintaining model reliability in live deployment.

4. Security

Given the sensitive nature of credit card data, strict security protocols must be followed to ensure data confidentiality and integrity. All datasets, especially real-world financial transaction data, must be anonymized to protect personal information. Additionally, the system must include secure data handling practices such as encrypted storage, secure APIs for model predictions, and access controls to prevent unauthorized access.

5.Usability

The system should be designed with end-users in mind, including analysts, developers, or risk officers. It should feature clear documentation, user-friendly interfaces (CLI, GUI, or dashboards), and easily interpretable outputs such as transaction-level predictions and model performance summaries. This ensures that the system can be adopted and used effectively with minimal training or technical support.

6.Maintainability

To support long-term usage and enhancements, the codebase should follow clean coding practices, be well-documented, and structured in a modular fashion. This makes it easy for developers to update or replace components such as adding new models, adjusting preprocessing steps, or integrating additional data sources. Version control, logging, and unit tests should be employed to facilitate collaborative development and system reliability.

4.7 FEASIBILITY STUDY

Preliminary investigation examines project feasibility; the likelihood the system will be useful to the organization. The main objective of the feasibility study is to test the Technical, Operational and Economical feasibility for adding new modules and debugging old running system. All systems are feasible if they are given unlimited resources and infinite time. There are aspects in the feasibility study portion of the preliminary investigation:

- Technical Feasibility
- Operation Feasibility
- Economical Feasibility

4.7.1 ECONOMIC FEASIBILITY:

The computerized system takes care of the present existing system's data flow and procedures completely and should generate all the reports of the manual system besides a host of other management reports.

It should be built as a web based application with separate web server and database server. This is required as the activities are spread throughout the organization customer wants a centralized database. Further some of the linked transactions take place in different locations.

Open source software like TOMCAT, JAVA, Mysql and Linux is used to minimize the cost for the Customer.

4.7.2Operational Feasibility

User-friendly

Customer will use the forms for their various transactions i.e. for adding new routes, viewing the routes details. Also the Customer wants the reports to view the various transactions based on the constraints. These forms and reports are generated as user-friendly to the Client.

Reliability

The package wills pick-up current transactions on line. Regarding the old

transactions, User will enter them in to the system.

Security

The web server and database server should be protected from hacking, virus etc

Portability

The application will be developed using standard open source software (Except Oracle) like Java, tomcat web server, Internet Explorer Browser etc these software will work both on Windows and Linux o/s. Hence portability problems will not arise.

Availability

This software will be available always.

Maintainability

The system uses the 2-tier architecture. The 1st tier is the GUI, which is said to be front-end and the 2nd tier is the database, which uses My-Sql, which is the back-end.

The front-end can be run on different systems (clients). The database will be running at the server. Users access these forms by using the user-ids and the passwords

4.7.3 Technical Feasibility

The technical issue usually raised during the feasibility stage of the investigation includes the following:

- Does the necessary technology exist to do what is suggested?
- Do the proposed equipments have the technical capacity to hold the data required to use the new system?
- Will the proposed system provide adequate response to inquiries, regardless of the number or location of users?
- Can the system be upgraded if developed?
- Are there technical guarantees of accuracy, reliability, ease of access and data security?

SYSTEM MODELS

4.8 SYSTEM MODELS

4.8.1 SCENARIOS

4.8.1.1 scenarios description for Bank Admin:

Scenario	Description	Expected Outcome
Register and login	Admin signs up and logs in	Successful authentication
View & authorize users	Review and approve registered users	User status updated
Add a new bank	Add bank details including image	Bank details saved
View fraud transactions (Random Forest)	Display frauds based on wrong CVV and expiry	List of flagged transactions
View product rank in chart	Chart visualization of product rankings	Chart displayed correctly

Table 4.8.1.1 scenarios for bank admin

4.8.1.2.Scenarios description for E-commerce Website User:

Scenario	Description	Expected Outcome
Register and login	E-commerce user signs up and logs in	User authenticated
Add category and products	Admin adds product category and items	Items added to inventory
View ranked products	View all products with ranking	Product list with rank displayed
View purchased products	Show all user purchases with total bill	Purchased history visible
View financial frauds	Identify fraudulent transactions	Fraud list displayed

Table 4.8.4.2 Scenarios for E-commerce Website User

4.8.1.3 Scenarios description for End User:

Scenario	Description	Expected Outcome
Register and login	End-user account creation and access	User logged in
Manage bank account	Update or view bank account details	Account managed
Request credit card	User requests new credit card	Request submitted
View credit card details	Access credit card number, CVV, expiry	Card details displayed
Transfer money	Add funds to the credit card	Transfer successful
Product search	Search for items by keywords	Search results shown
View purchase history	List all previous purchases and total cost	Purchases and bills displayed

Table 4.8.1.3 scenarios for End User

4.8.1.4 Scenarios description for Fraud Detection:

Scenario	Description	Expected Outcome
Apply AdaBoost	Boost performance of weak learners	Improved accuracy
Apply Majority Voting	Combine classifiers' decisions for final output	Majority vote prediction output
Add noise to data (10%, 20%, 30%)	Test system robustness with noisy data	Gradual degradation, still >90% accuracy
Hybrid Model Performance	Use AdaBoost with CNN, RNN, GBDT, etc.	Enhanced fraud detection results

Table 4.8.1.4 Scenarios for Fraud Detection

4.8.2 USE CASE DIAGRAM

A use case diagram at its simplest is a representation of a user's interaction with the system and depicting the specifications of a use case. A use case diagram can portray the different types of users of a system and the various ways that they interact with the system. This type of diagram is typically used in conjunction with the textual use case and will often be accompanied by other types of diagrams as well.

Overview:

Use case diagrams help model the functional requirements of a system from the user's perspective. In this context, we'll focus on the interactions between users (like fraud analysts or system admins) and the fraud detection system**.**

purpose:

1. **Visualize System Behaviour from a User's Perspective**:
   Use case diagrams help illustrate how users (actors) interact with a system, showing the functionality offered by the system in terms of user actions.

2. **Identify and Clarify Functional Requirements**:
   They make it easier to identify system functionalities (use cases) and how different users interact with them, which helps during requirement gathering.

3. **Simplify Communication with Stakeholders**:
   Since use case diagrams use simple visual symbols, they help both technical and non-technical stakeholders understand how the system works.

4. **Support System Design:**
   By outlining system functions and their interactions, use case diagrams support the design phase by identifying components/modules that need to be developed.

5. **Track Development Progress**:
   Each use case can represent a feature or functionality to be developed, making it useful for tracking implementation and testing.

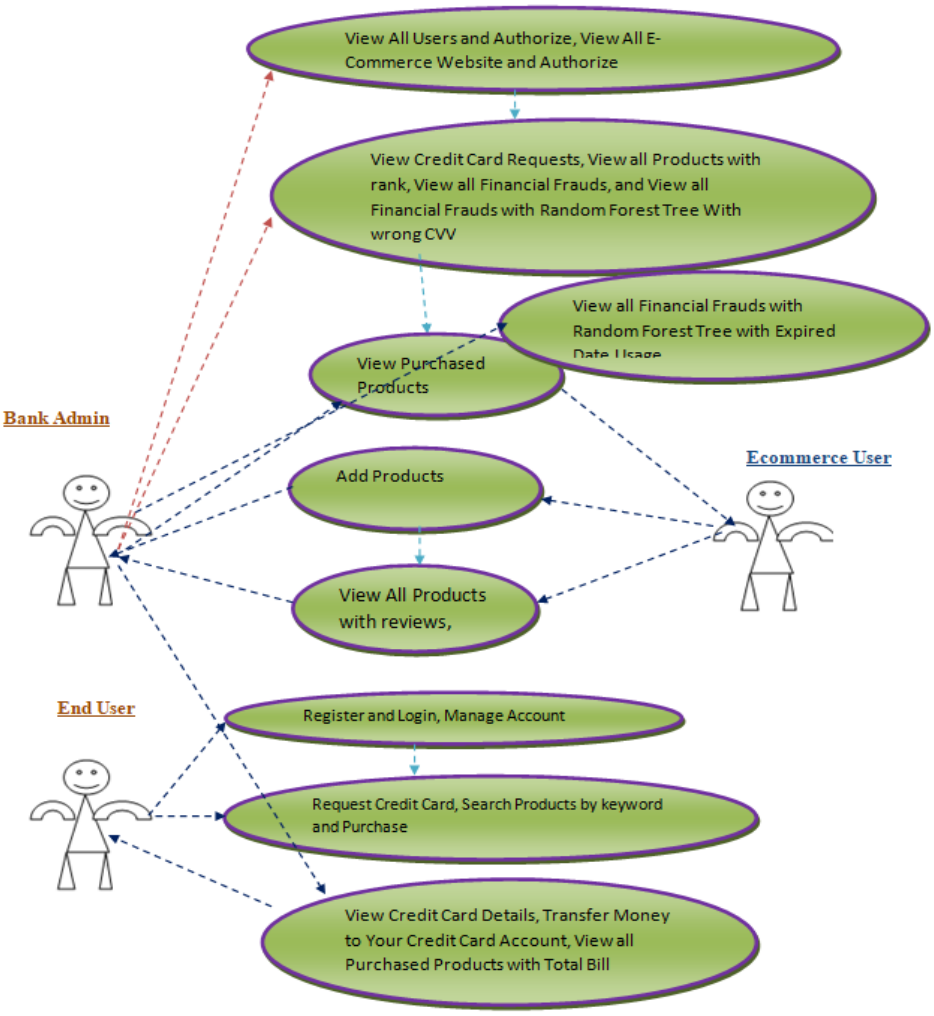


Fig4.8.2 Use Case Diagram

4.8.3 USE CASE DESCRIPTION

Use Case Description:

Credit Card Fraud Detection Using AdaBoost and Majority Voting

Primary Goal:

To develop an efficient fraud detection system that leverages machine learning, specifically AdaBoost and majority voting, to accurately identify fraudulent credit card transactions.

Actors Involved:

- Bank Admin – Manages users, financial fraud data, and system authorization.
- Ecommerce Website User – Adds products, views ranks, and monitors fraud data.
- End User – Interacts with the credit card system for registration, transactions, and viewing product data.

Preconditions:

- User registration must be completed and authorized.
- Dataset for credit card transactions is available.
- Machine learning models are trained on benchmark and real-world data.

BASIC FLOW

- User or Ecommerce site registers and is authorized by the Admin.
- Transactions are recorded, including relevant card and product details.
- Machine learning models (Naïve Bayes, Decision Tree, Random Forest, etc.) analyze the data.
- AdaBoost is applied to boost weak learners focusing on misclassified samples.
- Majority voting combines predictions of multiple models to finalize the classification (fraud or legitimate).
- Fraud cases are presented to the admin with details like wrong CVV, expired usage, etc.

Exceptions:

- Invalid input data may reduce model performance.
- Noise levels above 30% might affect model reliability.

Benefits:

- Enhanced detection accuracy using ensemble methods.
- Stable performance under noisy conditions.

- Easy integration with ecommerce and banking interfaces.
- Admin dashboard for visual monitoring of fraud trends.

Future Enhancements:

- Real-time fraud alerts using AI.
- Integration with mobile platforms.
- Deployment using cloud infrastructure for scalability.

4.8.4 OBJECT MODEL

4.8.4.1 CLASS DIAGRAM

The class diagram is the main building block of object oriented modeling. It is used both for general conceptual modeling of the systematic of the application, and for detailed modeling translating the models into programming code. Class diagrams can also be used for data modeling. The classes in a class diagram represent both the main objects, interactions in the application and the classes to be programmed. A class with three sections, in the diagram, classes is represented with boxes which contain three parts:

- The upper part holds the name of the class
- The middle part contains the attributes of the class
- The bottom part gives the methods or operations the class can take or undertake.

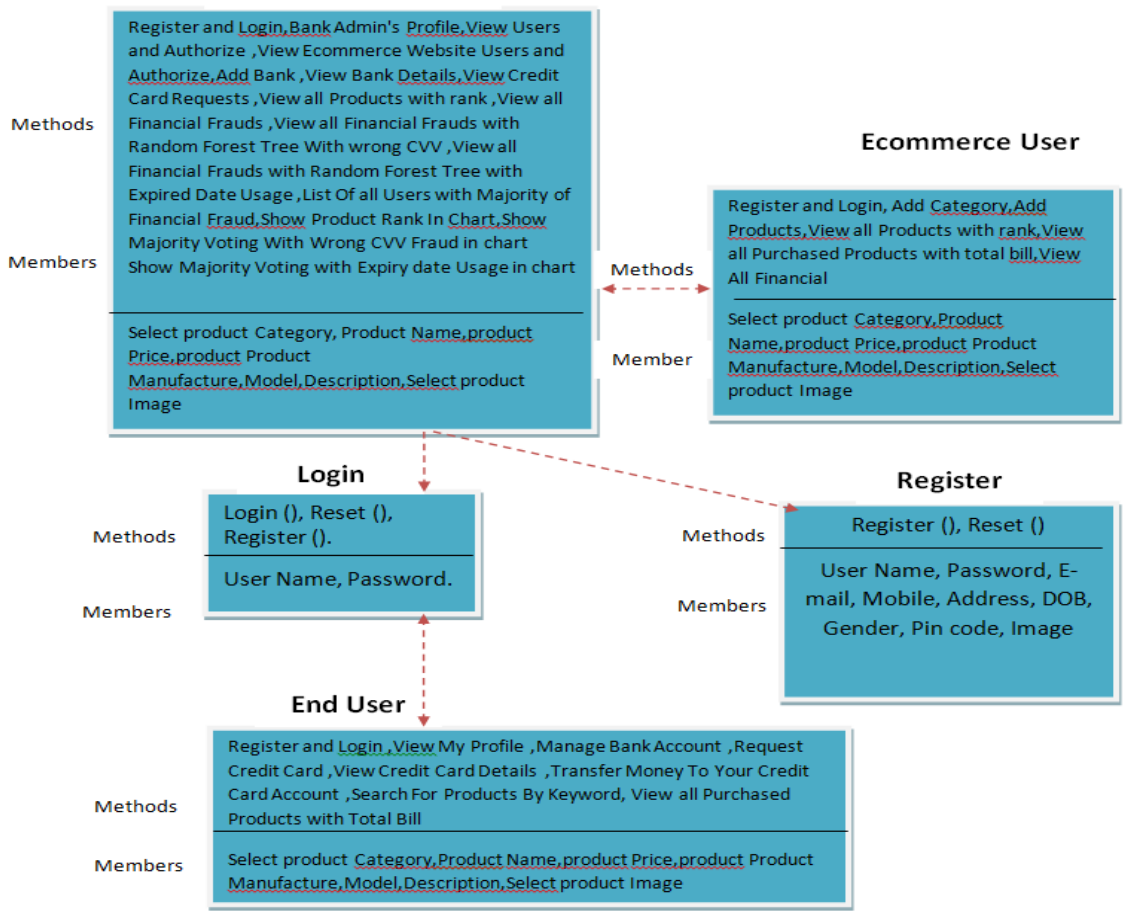


Fig4.8.4.1: Class Diagram.

4.8.5 DYNAMIC MODELS

4.8.5.1 SEQUENCE DIAGRAM

A sequence diagram is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. A sequence diagram shows object interactions arranged in time sequence. It depicts the objects and classes involved in the scenario and the sequence of messages exchanged between the objects needed to carry out the functionality of the scenario. Sequence diagrams are typically associated with use case realizations in the Logical View of the system under development. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

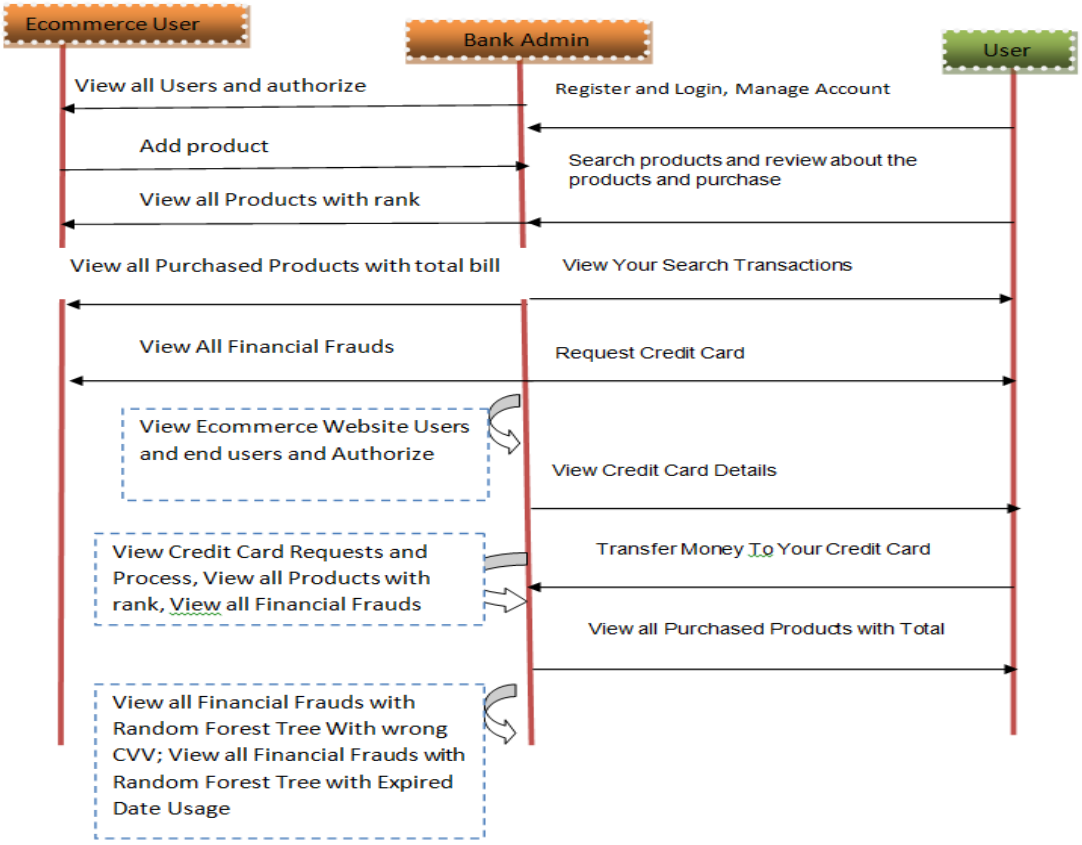


Fig4.8.5.1: Sequence diagram

4.8.5.2 COLLABORATION DIAGRAM

This UML Collaboration Diagram shows how different objects in a system interact to detect fraud in a transaction. The sequence numbers indicate the order of message exchanges.

Objects Involved:

1. EndUser – Initiates the transaction.
2. Transaction – Handles the transaction and triggers fraud detection.
3. MajorityVoting – Likely a fraud detection logic based on voting by models.
4. FraudReport – Prepares the fraud report.
5. BankAdmin – Receives the final fraud report.

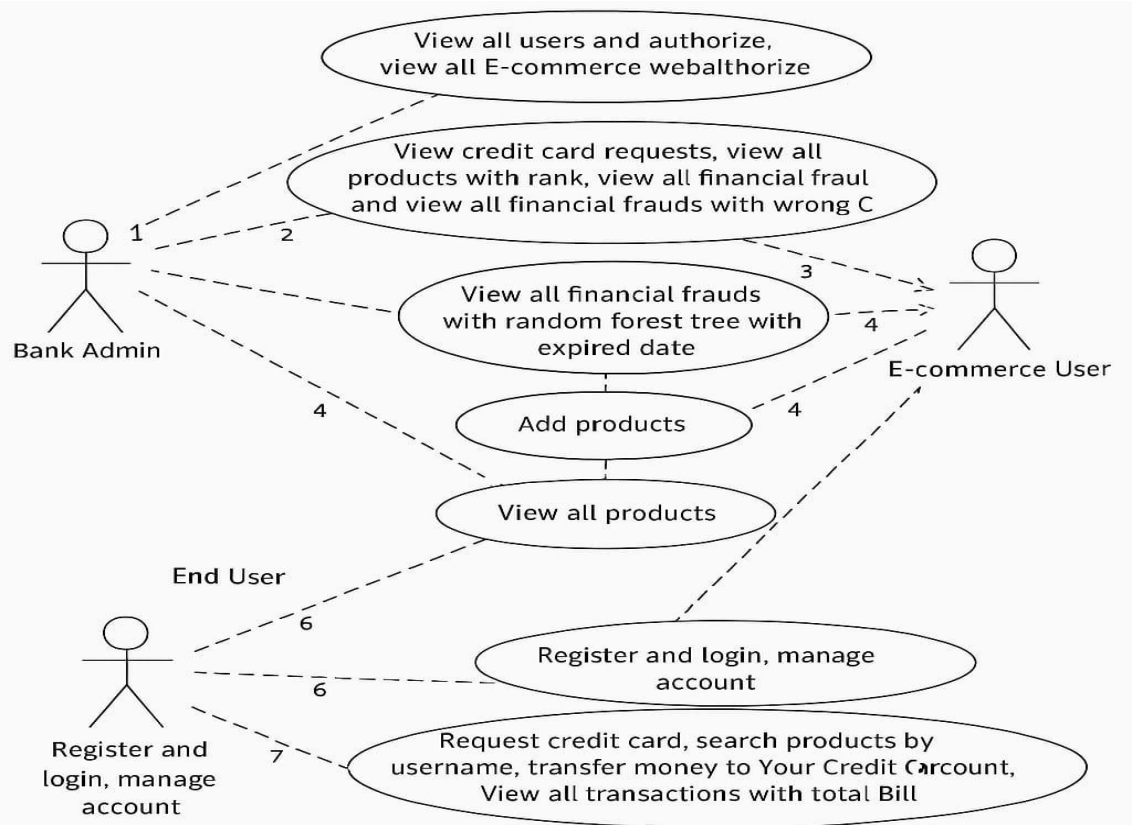


Fig 4.8.5.2:collaboration diagram

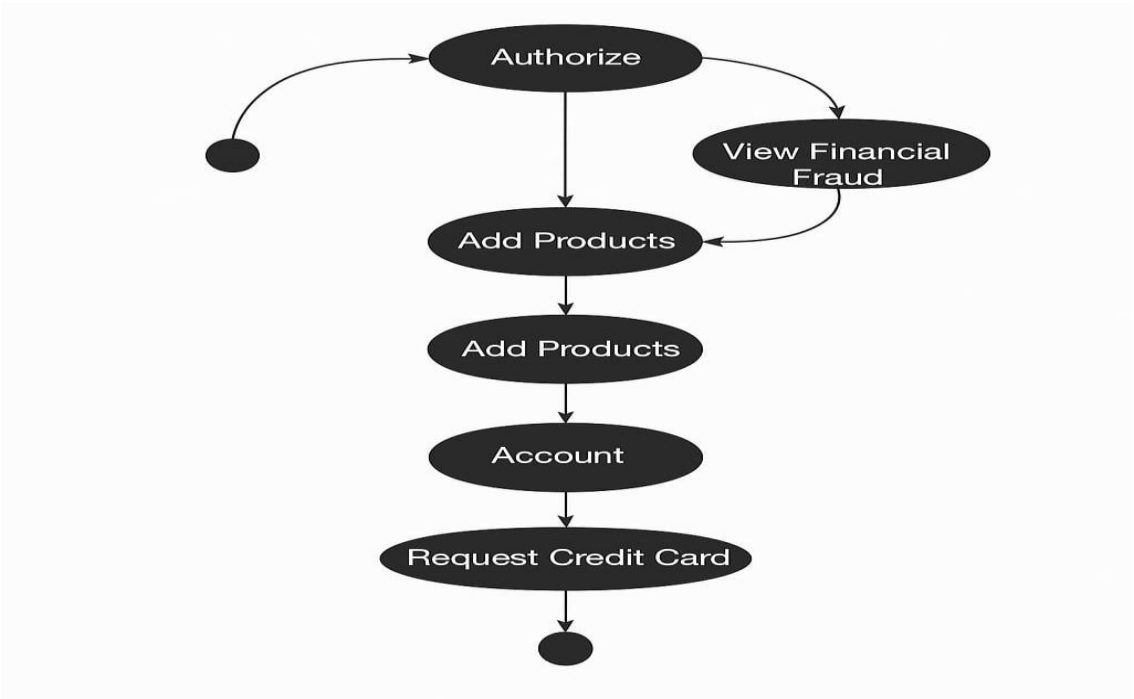
4.8.5.3 STATE CHART DIAGRAM

OVERVIEW:

A State Chart Diagram (also called a State Machine Diagram) is one of the behavioural UML diagrams that describes the dynamic behaviour of a system by showing its states, transitions, and events.

Purpose:

- To model the lifecycle of an object.
- To describe how an object responds to events depending on its current state.
- Especially useful for systems where the object behaviour is state-dependent, such as login/authentication flows, transaction processing**,** or UI navigation.



4.8.5.3 State Chart Diagram

4.8.5.4 ACTIVITY DIAGRAM

An activity diagram is a type of UML (Unified Modeling Language) diagram that visually represents the flow of control or data within a system or process. It is especially useful for modeling the dynamic aspects of a system, such as workflows, operations, or business processes.

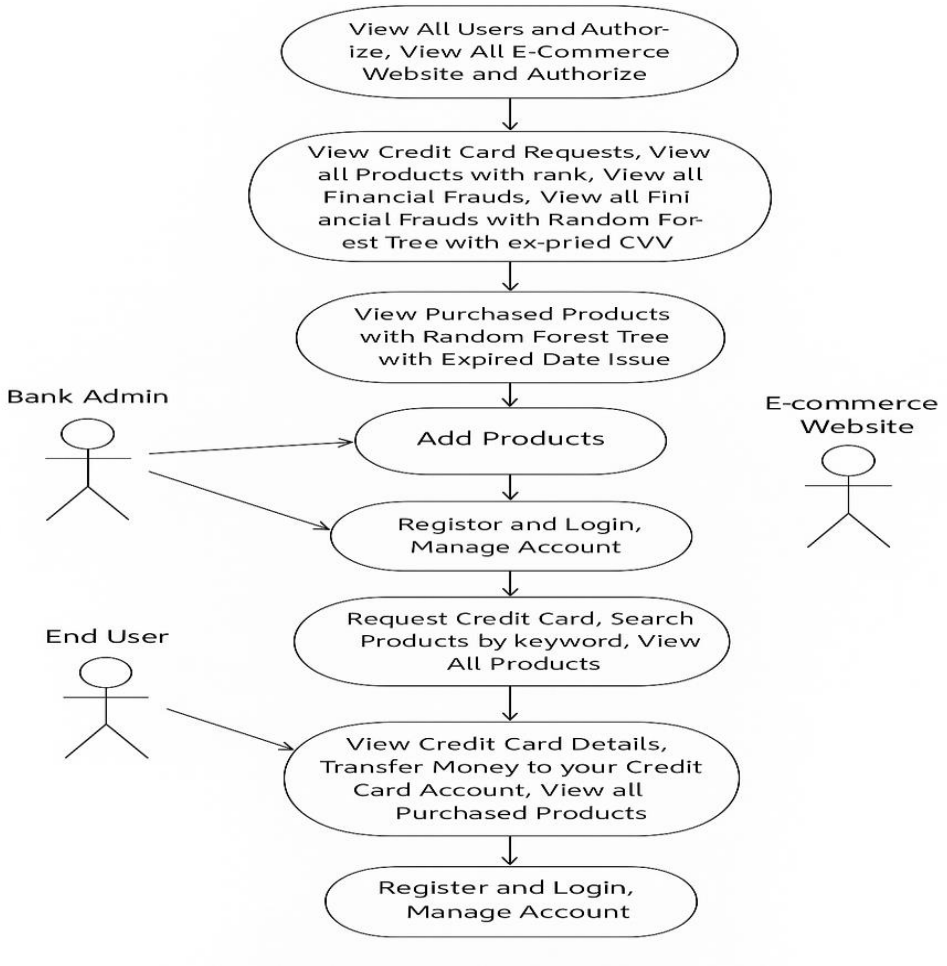


Fig 4.8.5.4 Activity Diagram

4.8.5.5 FLOW CHART

4.8.5.1 FLOW CHART 1:USER

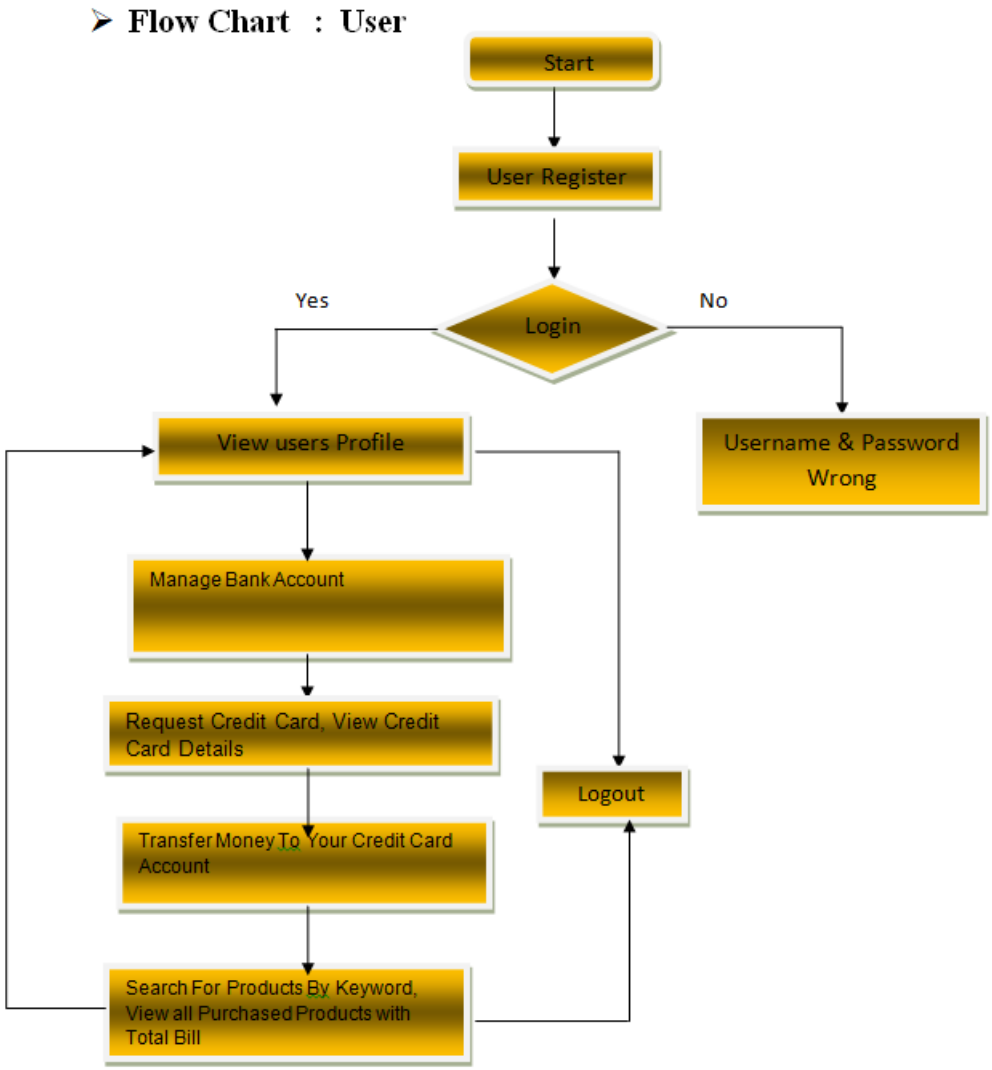


Fig 4.8.5.1 User

4.8.5.2 FLOW CHART 2:BANK ADMIN:

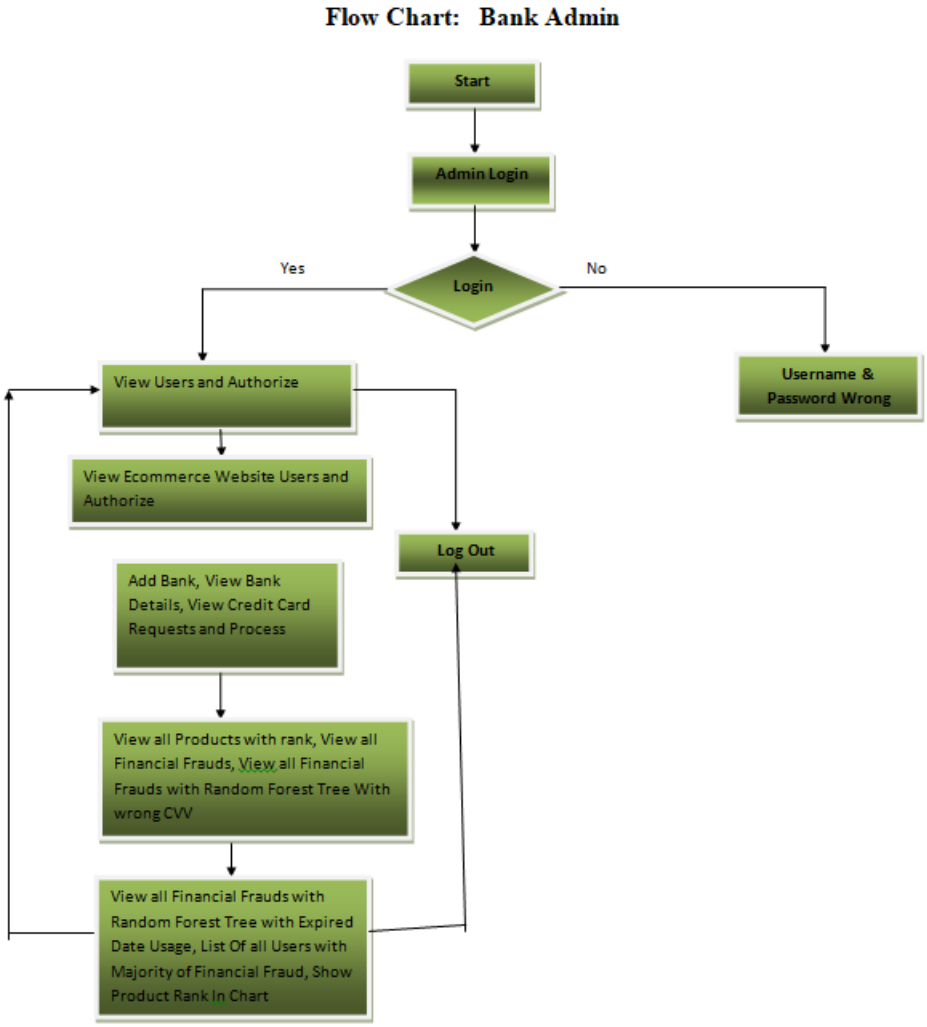


Fig 4.8.5.2 Bank Admin

4.8.5.3 FLOW CHART 3:E-COMMERCE USER:

➢ **Flow Chart : Ecommerce User**

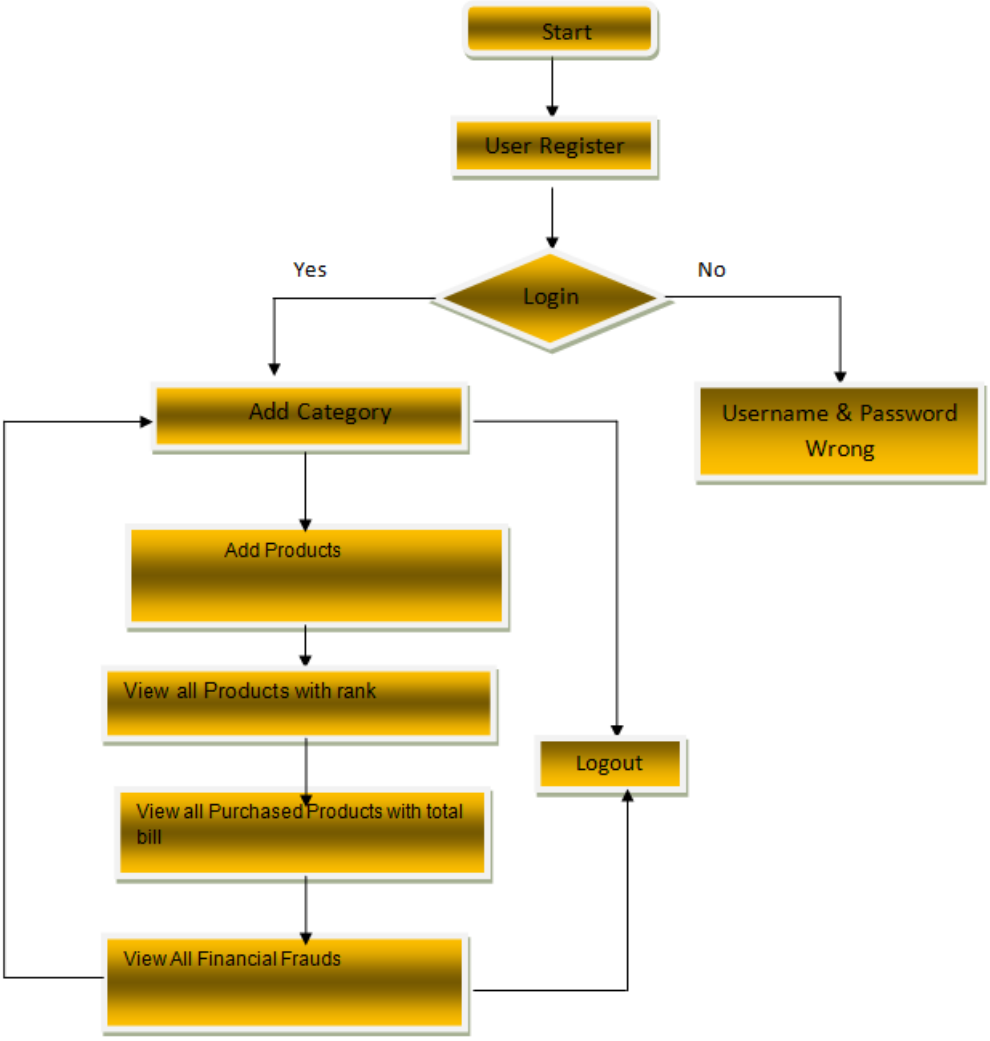


Fig 4.8.5.3 E-Commerce User

4.8.5.6 COMPONENT DAIGRAM

The component diagram outlines the high-level architecture and interaction between various components of the credit card fraud detection system. The goal is to visualize how different parts collaborate to identify fraudulent transactions.

Main Components:

1. **User Interface (UI):**
   - Allows users (like administrators or analysts) to interact with the system.
   - Input and visualization of transaction data and fraud results.
2. **Core System Component:**
   - The central part of the system that handles coordination between other modules.
   - Responsible for passing data between the UI, database, and detection modules.
3. **Database:**
   - Stores transaction data, user profiles, training data, and fraud records.
   - Acts as the data source for the system.
4. **AdaBoost Module:**
   - A machine learning component that builds strong classifiers from multiple weak classifiers.
   - Used to increase detection accuracy.
5. **Majority Voting Module:**
   - Ensemble technique that combines predictions from multiple classifiers.
   - Determines final fraud decision based on majority rule.
6. **Credit Card Fraud Detection System:**
   - The top-level system that integrates all sub-components.
   - Provides final fraud classification results.

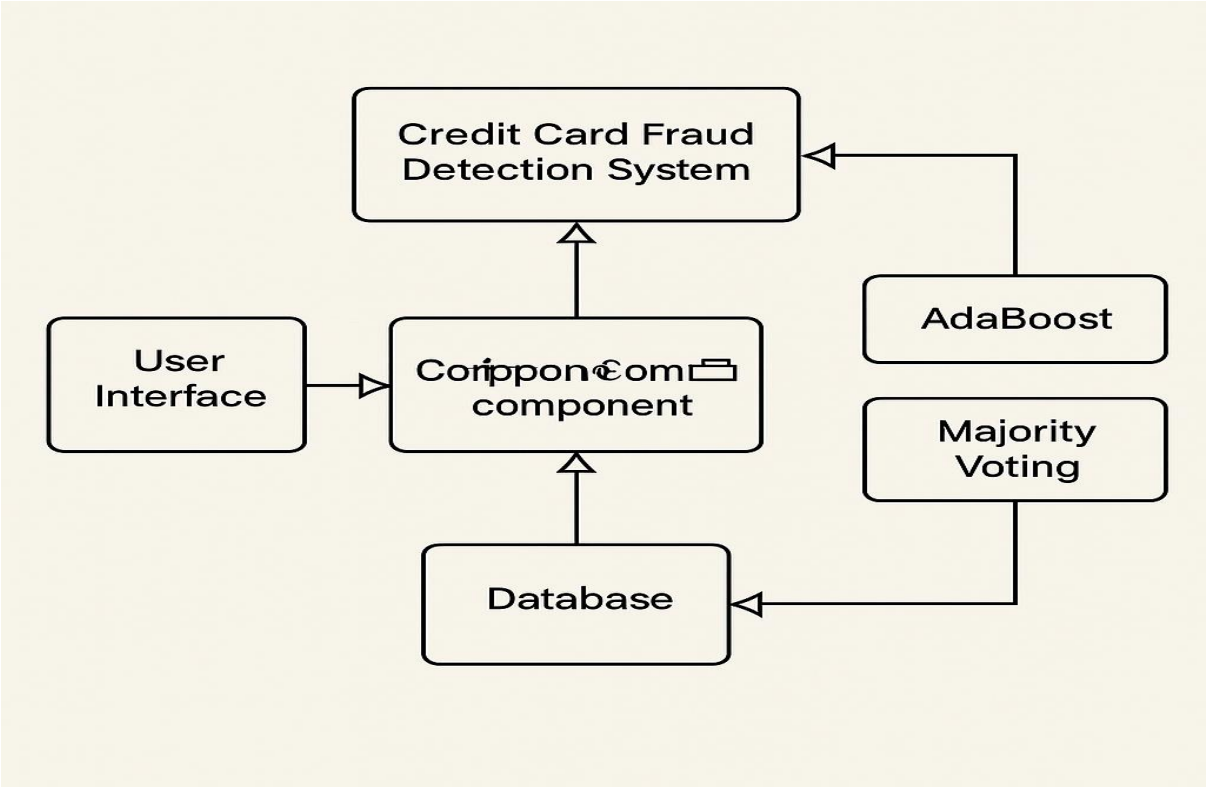


Fig 4.8.5.6 Component Diagram

5.SYSTEM DESIGN

5.1 SUBSYSTEM SPECIFICATION

The system is divided into the following subsystems for better modularity and maintainability:

1. User Management Subsystem

Purpose:

To manage user-related operations including registration, authentication, and authorization for Admins, E-commerce users, and End Users.

Key Functionalities:

- User Registration and Login
- User Role Assignment (Admin, E-commerce, End User)
- Account management (view/edit profile)
- Authorization by Admin

2. Fraud Detection Subsystem

Purpose:

Implements fraud detection logic using machine learning algorithms including AdaBoost and Majority Voting mechanisms.

Key Functionalities:

- Input of credit card transaction data
- Feature extraction and preprocessing
- Execution of base classifiers (e.g., NB, DT, SVM)
- Hybrid classification using AdaBoost
- Ensemble classification using Majority Voting
- Performance evaluation with and without added noise

3. Admin Control Subsystem

Purpose:

To allow the administrator to manage users, banks, fraud reports, and system visualizations.

Key Functionalities:

- Authorize registered users
- View/Add/Edit/Delete Bank details
- View credit card requests and financial fraud reports
- View fraud detection results via Random Forest and AdaBoost classifiers
- Display fraud analytics and charts (Wrong CVV, Expired Card Usage)

4. E-commerce Management Subsystem

Purpose:

To manage product categories and product listings for users.

Key Functionalities:

- Add/Edit/Delete product categories
- Add/Edit/Delete products
- View ranked products
- View purchased product history

5. End User Interaction Subsystem

Purpose:

Allows end users to interact with the system for purchasing products, managing accounts, and requesting credit cards.

Key Functionalities:

- Credit card application and account management
- Transaction execution (purchase and payments)
- View and search product listings
- View transaction history and fraud alerts

6. Reporting and Visualization Subsystem

Purpose:

Generate charts and reports based on system and fraud detection data for better decision-making and insights.

Key Functionalities:

- Generate pie/bar charts for fraud type distributions

- Show ranked products and fraud statistics
- Export fraud detection results and reports

7. Data Storage and Database Management Subsystem

Purpose:

Handles the data layer of the application using MySQL database.

Key Functionalities:

- User and admin account storage
- Transaction logs and product data storage
- Fraud detection logs
- Model training data and result persistence

5.2 SYSTEM ARCHITECTURE

The purpose of the design phase is to arrange an answer of the matter such as by the necessity document. This part is that the opening moves in moving the matter domain to the answer domain. The design phase satisfies the requirements of the system. The design of a system is probably the foremost crucial issue warm heartedness the standard of the software package. It's a serious impact on the later part, notably testing and maintenance.

The output of this part is that the style of the document. This document is analogous to a blueprint of answer and is employed later throughout implementation, testing and maintenance. The design activity is commonly divided into 2 separate phases System Design and Detailed Design.

System Design conjointly referred to as top-ranking style aims to spot the modules that ought to be within the system, the specifications of those modules, and the way them move with one another to supply the specified results.

At the top of the system style all the main knowledge structures, file formats, output formats, and also the major modules within the system and their specifications square measure set. System design is that the method or art of process the design, components, modules, interfaces, and knowledge for a system to satisfy such as needs. Users will read it because the application of systems theory to development.

Detailed Design, the inner logic of every of the modules laid out in system design is determined. Throughout this part, the small print of the info of a module square measure sometimes laid out in a high-level style description language that is freelance of the target language within which the software package can eventually be enforced.

In system design the main target is on distinguishing the modules, whereas throughout careful style the main target is on planning the logic for every of the modules.

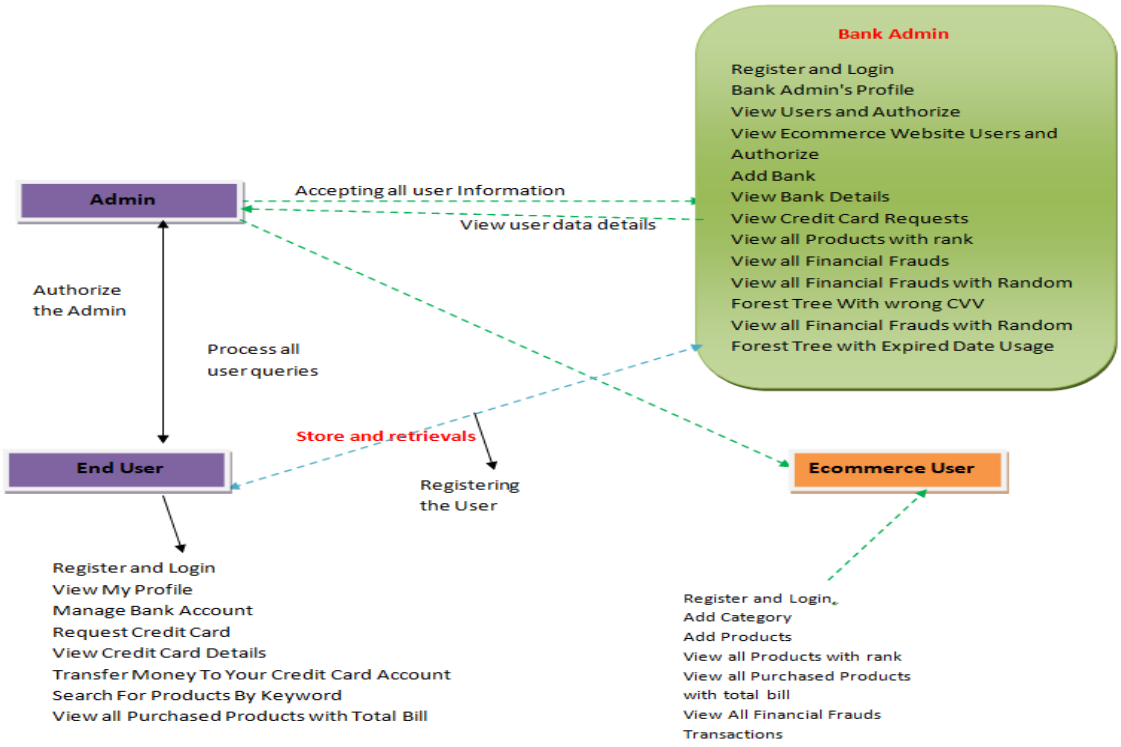


Fig5.2: Architecture diagram

6.CODING

6.1SAMPLE CODE

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">

<head>

<title>Adding Bank Details Page</title>

<meta http-equiv="Content-Type" content="text/html; charset=utf-8" />

<link href="css/style.css" rel="stylesheet" type="text/css" />

<link rel="stylesheet" type="text/css" href="css/coin-slider.css" />

<script type="text/javascript" src="js/cufon-yui.js"></script>

<script type="text/javascript" src="js/cufon-quicksand.js"></script>

<script type="text/javascript" src="js/jquery-1.4.2.min.js"></script>

<script type="text/javascript" src="js/script.js"></script>

<script type="text/javascript" src="js/coin-slider.min.js"></script>

<style type="text/css">

<!--

.style1 {

        font-size: 24px;

        color: #FF0000;

}

.style3 {color: #333333}

.style4 {color: #CC6600}

.style5 {color: #660000}

.style7 {color:#006666}

-->

</style>
```

```
</head>
<body>
<div class="main">
  <div class="header">
    <div class="header_resize">
      <div class="logo">
        <h1 align="center"><a href="index.html" class="style1">Credit card fraud
detection using AdaBoost and majority voting</a></h1>
      </div>
      <div class="menu_nav">
        <ul>
          <li class="active"><a href="index.html"><span>Home Page</span></a></li>
          <li><a href="AdminLogin.jsp"><span>Bank Admin</span></a></li>
          <li><a href="EcommerceLogin.jsp"><span>Ecommerce</span></a></li>
          <li><a href="UserLogin.jsp"><span>User</span></a></li>
          <li><a href="#"><span>Contact Us</span></a></li>
        </ul>
      </div>
      <div class="clr"></div>
      <div class="slider">
        <div id="coin-slider"> <a href="#"><img src="images/slide1.jpg" width="935"
height="285" alt="" /> </a> <a href="#"><img src="images/slide2.jpg" width="935"
height="285" alt="" /> </a> <a href="#"><img src="images/slide3.jpg" width="935"
height="285" alt="" /> </a> </div>
        <div class="clr"></div>
      </div>
      <div class="clr"></div>
    </div>
  </div>
```

```
 <div class="content">
   <div class="content_resize">
     <div class="mainbar">
       <div class="article">
         <h2 class="style4" style="color:#CC6600">Adding Bank Details..</h2>
         <div class="clr"></div>
         <div class="post_content">
           <p> </p>
         <form action="Admin_AddBank1.jsp" method="post" enctype="multipart/form-
data" id="s" ">
 <%


try{
                                                                    String
bank=(String)application.getAttribute("adbank");
                                                                    %>
                 <table width="500" border="0" align="center">
                   <tr>
                     <td width="146" height="40"><div align="justify"><span class="style8
style38 style3"> Bank Name </span> </div></td>
                     <td width="277"><label>
                       <input type="text" id="t1" name="bname" value="<%=bank%>"  />
                     </label></td>
                   </tr>
                                        <tr>
                     <td height="60"><div align="justify"><span class="style3 style8
style38">Bank Address</span></div></td>
                     <td><label>
```

```
<textarea name="address" cols="19" rows="2"
id="textarea"></textarea>

</label></td>

</tr>

<tr>

<td height="40"><div align="justify"><span class="style3 style8
style38">Bank Location </span></div></td>

<td><label>

<input type="text" id="t3" name="loc" />

</label></td>

</tr>

<tr>

<td height="40"><div align="justify"><span class="style3 style8
style38">Bank Pincode </span></div></td>

<td><label>

<input type="text" id="t3" name="pin" />

</label></td>

</tr>

<tr>

<td height="40"><div align="justify"><span class="style3 style8
style38">Bank Contact No </span></div></td>

<td><label>

<input type="text" id="t3" name="cno" />

</label></td>

</tr>

<tr>

<td height="40"><div align="justify"><span class="style3 style8
style38">Bank Email Id </span></div></td>

<td><label>
```

```
<input type="text" id="t3" name="mail" />
</label></td>
</tr>
<tr>
<td height="40"><div align="justify"><span class="style3 style8
style38"> Select Building Image </span></div></td>
<td><input type="file" id="bankImage" name="image"
style="width:100%" /></td>
</tr>
<tr>
<td> </td>
<td> </td>
</tr>
<tr>
<td height="26"> </td>
<td>

<div align="left">
<input type="submit" name="Submit" value="Add Bank" />
</div></td>
</tr>
</table>
<%
}
catch(Exception e)
{
out.print(e);
}
```

```
                                        %>
        </form>
                                <p> </p>


                                        <div align="left"><a href="AdminMain.jsp"
class="style11"><strong>Back</strong></a></div>


        </div>
        <div class="clr"></div>
      </div>
      </div>
     <div class="sidebar">
      <div class="searchform">
        <form id="formsearch" name="formsearch" method="post" action="#">
          <span>
          <input name="editbox_search" class="editbox_search" id="editbox_search"
maxlength="80" value="Search our ste:" type="text" />
          </span>
          <input name="button_search" src="images/search.gif" class="button_search"
type="image" />
        </form>
      </div>
      <div class="clr"></div>
      <div class="gadget">
        <h2 class="star"><span>Sidebar</span> Menu</h2>
        <div class="clr"></div>
        <ul class="sb_menu">
          <li><a href="AdminMain.jsp">Home</a></li>
```

```
                    <li><a href="AdminLogin.jsp">Logout</a></li>


        </ul>
       </div>


     </div>
     <div class="clr"></div>
    </div>
  </div>
  <div class="footer"></div>
</div>
<div align=center></div>
</body>
</html>


<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<%@ include file="connect.jsp" %>
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<title>Authorize Users</title>
<meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
<link href="css/style.css" rel="stylesheet" type="text/css" />
<link rel="stylesheet" type="text/css" href="css/coin-slider.css" />
```

```
<script type="text/javascript" src="js/cufon-yui.js"></script>
<script type="text/javascript" src="js/cufon-quicksand.js"></script>
<script type="text/javascript" src="js/jquery-1.4.2.min.js"></script>
<script type="text/javascript" src="js/script.js"></script>
<script type="text/javascript" src="js/coin-slider.min.js"></script>
<style type="text/css">
<!--
.style1 {
        font-size: 24px;
        color: #FF0000;
}
.style4 {color: #CC6600}
.style5 {color: #660000}
.style7 {color: #000000;
                font-size: 14px}
.style8 {font-weight: bold}
.style9 {color:#996600;
                font-size: 14px}
.style10 {color:#FF3333;
                font-size: 14px}
.style11 {font-size: 12px}
-->
</style>
</head>
<body>
<div class="main">
  <div class="header">
```

```
    <div class="header_resize">
      <div class="logo">
        <h1 align="center"><a href="index.html" class="style1">Credit card fraud
detection using AdaBoost and majority voting</a></h1>
      </div>
      <div class="menu_nav">
        <ul>
          <li class="active"><a href="index.html"><span>Home Page</span></a></li>
          <li><a href="AdminLogin.jsp"><span>Bank Admin</span></a></li>
          <li><a href="EcommerceLogin.jsp"><span>Ecommerce</span></a></li>
          <li><a href="UserLogin.jsp"><span>User</span></a></li>
          <li><a href="#"><span>Contact Us</span></a></li>
        </ul>
      </div>
      <div class="clr"></div>
      <div class="slider">
        <div id="coin-slider"> <a href="#"><img src="images/slide1.jpg" width="935"
height="285" alt="" /> </a> <a href="#"><img src="images/slide2.jpg" width="935"
height="285" alt="" /> </a> <a href="#"><img src="images/slide3.jpg" width="935"
height="285" alt="" /> </a> </div>
        <div class="clr"></div>
      </div>
      <div class="clr"></div>
    </div>
  </div>
  <div class="content">
    <div class="content_resize">
      <div class="mainbar">
        <div class="article">
```

```
<h2 class="style4" style="color:#CC6600">Authorize Ecommerce Website Users..</h2>

<div class="clr"></div>

<div class="post_content">

  <p> </p>

        <table width="570" border="1" align="left"  cellpadding="0" cellspacing="0"  ">

      <tr>

        <td  width="35"  valign="middle" height="34" style="color: #2c83b0;"><div align="center" class="style57 style56 style5"><b>ID</b></div></td>

        <td  width="47"  valign="middle" height="34" style="color: #2c83b0;"><div align="center" class="style57 style56 style5"><b>User Name</b></div></td>

                                <td  width="47"  valign="middle" height="34" style="color: #2c83b0;"><div align="center" class="style57 style56 style5"><b>Transport Company</b></div></td>

        <td  width="47"  valign="middle" height="34" style="color: #2c83b0;"><div align="center" class="style57 style56 style5"><b>Email</b></div></td>

        <td  width="47"  valign="middle" height="34" style="color: #2c83b0;"><div align="center" class="style57 style56 style5"><b>Address</b></div></td>

        <td  width="47"  valign="middle" height="34" style="color: #2c83b0;"><div align="center" class="style57 style56 style5"><b>Status</b></div></td>

      </tr>

      <%


                                String s1,s2,s3,s4,s5,s6,s7;

                                int i=0;

                                try

          {
```

```
String query="select * from euser";
Statement st=connection.
createStatement();
ResultSet
rs=st.executeQuery(query);
while ( rs.next() )
{
        i=rs.getInt(1);
        s1=rs.getString(3);
        s2=rs.getString(2);
        s3=rs.getString(5);
        s4=rs.getString(7);
        s5=rs.getString(10);
%>
        <tr>
          <td height="0" align="center"  valign="middle"><div align="center"
class="style7">
            <%out.println(i);%>
          </span></div></td>
          <td height="0" align="center"  valign="middle"><div align="center"
class="style10">
              <span class="style10">
```

```
                    <b><%out.println(s1);%></b>

                    </span></div></td>

                                        <td height="0" align="center"
valign="middle"><div align="center" class="style9">

                    <span class="style9">

                    <b><%out.println(s2);%></b>

                    </span></div></td>

                <td height="0" align="center"  valign="middle"><div align="center"
class="style7">



                    <%out.println(s3);%>

                    </span></div></td>

                <td height="0" align="center"  valign="middle"><div align="center"
class="style7">



                    <%out.println(s4);%>

                    </span></div></td>

                <%

                                        if(s5.equalsIgnoreCase("waiting"))

                                        {



                                        %>

                <td valign="middle" height="0"
style="color:#000000;"align="center"><div align="center" class="style22 style5
style20 style30 style37">

                  <div align="center" class="style2"><a
href="Admin_UserStatus.jsp?type=<%="elogin"%>&id=<%=i%>"
class="style5">waiting</a></div>

                </div></td>

                <%
```

```
                                        }
                                        else
                                        {
                                        %>
                <td width="17" height="0" align="center"  valign="middle"><div
align="center" class="style7">
                  <b><%out.println(s5);%></b>
                </div></td>
                <%
                                        }


                                        %>
              </tr>
              <%
                                        }




                                        connection.close();
                                }
                                catch(Exception e)
                                {
                                        out.println(e.getMessage());
                                }
                                %>
        </table>
                                <p class="style19"> </p>
```

```
                    <p> </p>
                                <table width="476" border="0" cellspacing="2"
cellpadding="2">
        <tr>
          <td width="468"><div align="left"><a href="AdminMain.jsp"
class="style95">Back</a></div></td>
        </tr>
       </table>



       </div>
       <div class="clr"></div>
      </div>
      </div>
     <div class="sidebar">
      <div class="searchform">
       <form id="formsearch" name="formsearch" method="post" action="#">
        <span>
        <input name="editbox_search" class="editbox_search" id="editbox_search"
maxlength="80" value="Search our ste:" type="text" />
        </span>
        <input name="button_search" src="images/search.gif" class="button_search"
type="image" />
       </form>
      </div>
     <div class="clr"></div>
      <div class="gadget">
```

```
        <h2 class="star"><span>Sidebar</span> Menu</h2>
        <div class="clr"></div>
        <ul class="sb_menu">
          <li><a href="AdminMain.jsp">Home</a></li>
                        <li><a href="AdminLogin.jsp">Logout</a></li>
        </ul>
      </div>
    </div>
    <div class="clr"></div>
  </div>
 </div>
 <div class="footer"></div>
</div>
<div align=center></div>
</body>
</html>
```

6.2 SAMPLE SCREENS

6.2.1HOME PAGE

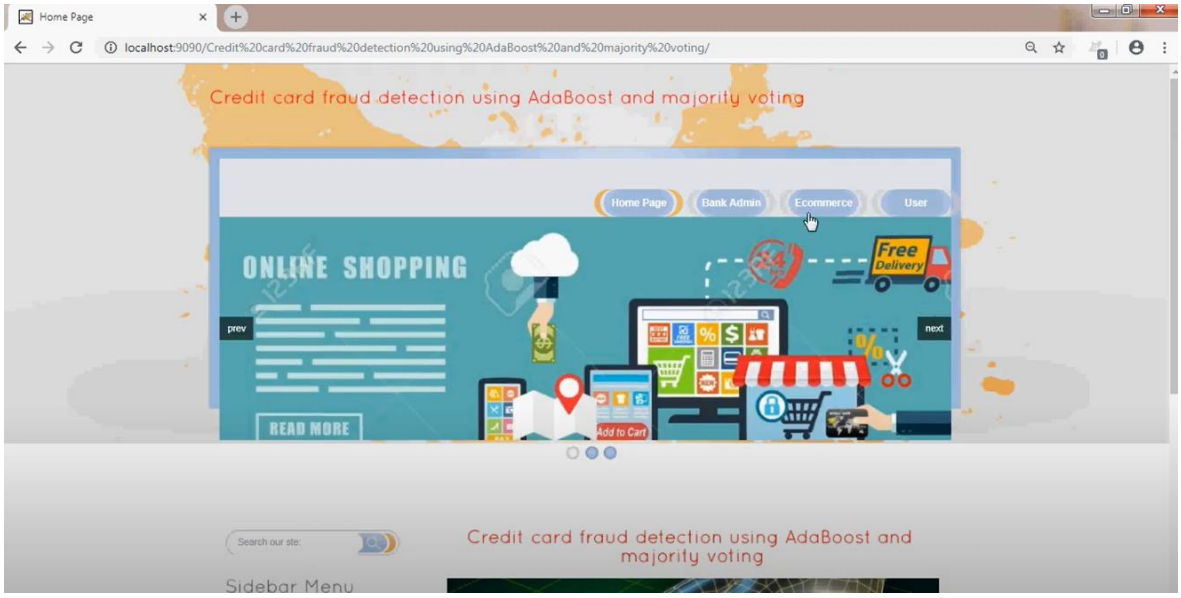


Fig 6.2.1 Home page

6.2.2 BANK ADMIN REGISTRATION PAGE

Fig 6.2.3 Bank Admin Registration Page

6.2.3BANK ADMIN LOGIN PAGE

Fig 6.2.4 Bank Admin Login Page

6.2.4 E-COMMERCE USER REGISRATION

6.2.4 E-Commerce User Registration page

6.2.5 E-COMMERCE USER LOGIN

Fig 6.2.5 E-Commerce User Login Page

6.2.6 USER REGISTRATION

Fig 6.2.6 User Registration Page

7.TESTING

7.1 SYSTEM TESTING

Testing has become an integral part of any system or project especially in the field of information technology. The importance of testing is a method of justifying, if one is ready to move further, be it to be check if one is capable to with stand the rigors of a particular situation cannot be underplayed and that is why testing before development is so critical. When the software is developed before it is given to user to user the software must be tested whether it is solving the purpose for which it is developed. This testing involves various types through which one can ensure the software is reliable. The program was tested logically and pattern of execution of the program for a set of data are repeated. Thus the code was exhaustively checked for all possible correct data and the outcomes were also checked.

8.2 MODULE TESTING

To locate errors, each module is tested individually. This enables us to detect error and correct it without affecting any other modules. Whenever the program is not satisfying the required function, it must be corrected to get the required result. Thus all the modules are individually tested from bottom up starting with the smallest and lowest modules and proceeding to the next level. Each module in the system is tested separately. For example the job classification module is tested separately. This module is tested with different job and its approximate execution time and the result of the test is compared with the results that are prepared manually. Each module in the system is tested separately. In this system the resource classification and job scheduling modules are tested separately and their corresponding results are obtained which reduces the process waiting time.

8.3 INTEGRATION TESTING

After the module testing, the integration testing is applied. When linking the modules there may be chance for errors to occur, these errors are corrected by using this testing. In this system all modules are connected and tested. The testing results are very correct. Thus the mapping of jobs with resources is done correctly by the system

8.4 ACCEPTANCE TESTING

When that user fined no major problems with its accuracy, the system passers through a final acceptance test. This test confirms that the system needs the original goals, objectives and requirements established during analysis without actual execution which elimination wastage of time and money acceptance tests on the shoulders of users and management, it is finally acceptable and ready for the operation.

7.2 TEST CASES

7.2.1 Bank Admin:

Test Case Description	Expected Outcome
Register a new Bank Admin account	Successful
Login with valid credentials	Successful
View Bank Admin's Profile	Profile details
View Users and Authorize	User list
View Ecommerce Website Users and Authorize	Website user list
Add Bank	Bank added
View Bank Details	Bank details
View Credit Card Requests	Requests list
View all Products with rank	Products list
View all Financial Frauds	Fraud list
View all Financial Frauds with Random Forest Tree With wrong CVV	Fraud list
View all Financial Frauds with Random Forest Tree with Expired Date Usage	Fraud list
List Of all Users with Majority of Financial Fraud	User list
Show Product Rank In Chart	Chart displayed
Show Majority Voting With Wrong CVV Fraud in chart	Chart displayed
Show Majority Voting with Expiry date Usage in chart	Chart displayed

Table 7.2.1 Bank Admin

7.2.2 E-commerce Website:

Test Case Description	Expected Outcome
Register a new E-commerce Website user account	Successful
Login with valid credentials	Successful
Add Category	Category added
Add Products	Product added
View all Products with rank	Products list
View all Purchased Products with total bill	Purchased list
View All Financial Frauds	Fraud list

Table 7.2.2 E-commerce Website

7.2.3 End User:

Test Case Description	Expected Outcome
Register a new End User account	Successful
Login with valid credentials	Successful
View My Profile	Profile details
Manage Bank Account	Bank account managed
Request Credit Card	Credit card requested
View Credit Card Details	Credit card details
Transfer Money To Your Credit Card Account	Successful
Search For Products By Keyword	Products list
View all Purchased Products with Total Bill	Purchased list

8 .USER MANUAL

Software's To Be Installed

1. Install Java (JDK) SE Developmem kit 1.6.0(jdk1.6.0)

2. Install MySQL

3. Install Tomcat 6.0

5. Copy your project (Social Network Rumor Diffusion Predication Based. On Equal Responsibility Game model) and your project corresponding databases in C:\Program Files (x86)\Apache Software Foundation Tomcat 6.0\web apps with some name like web media and stock market

Steps To Run the Project

1.Open Webserver Tomcat 6.0 from web browser by giving http://localhost:8080 (port nuber:8080)

2.. Now Click on the tomcat manager viewed in path column in application table then open it.

3. Then login to tomcat app with credentials.

4.Then it shows the tomcat web application manager in applications list it shows project title we have to click on the project title

5. Then your project runs successfully.

9.CONCLUSION

A study on credit card fraud detection using machine learning algorithms has been presented in this paper. A number of standard models which include NB, SVM, and DL have been used in the empirical evaluation. A publicly available credit card data set has been used for evaluation using individual (standard) models and hybrid models using AdaBoost and majority voting combination methods. The MCC metric has been adopted as a performance measure, as it takes into account the true and false positive and negative predicted outcomes. The best MCC score is 0.823, achieved using majority voting. A real credit card data set from a financial institution has also been used for evaluation. The same individual and hybrid models have been employed. A perfect MCC score of 1 has been achieved using AdaBoost and majority voting methods. To further evaluate the hybrid models, noise from 10% to 30% has been added into the data samples. The majority voting method has yielded the best MCC score of 0.942 for 30% noise added to the data set. This shows that the majority voting method is stable in performance in the presence of noise.

Future Enhancements:

It is not possible to develop a system that makes all the requirements of the user. User requirements keep changing as the system is being used. Some of the future enhancements that can be done to this system are:

- As the technology emerges, it is possible to upgrade the system and can be adaptable to desired environment.

BIBLIOGRAPHY

10. BIBLIOGRAPHY

[1] Y. Sahin, S. Bulkan, and E. Duman, “A cost-sensitive decision tree approach for fraud detection,” Expert Systems with Applications, vol. 40, no. 15, pp. 5916–5923, 2013.

[2] A. O. Adewumi and A. A. Akinyelu, “A survey of machine-learning and nature-inspired based credit card fraud detection techniques,” International Journal of System Assurance Engineering and Management, vol. 8, pp. 937–953, 2017.

[3] A. Srivastava, A. Kundu, S. Sural, A. Majumdar, “Credit card fraud detection using hidden Markov model,” IEEE Transactions on Dependable and Secure Computing, vol. 5, no. 1, pp. 37–48, 2008.

[4] The Nilson Report (October 2016) [Online]. Available: https://www.nilsonreport.com/upload/content_promo/The_Nilson_R eport_10-17-2016.pdf

[5] J. T. Quah, and M. Sriganesh, “Real-time credit card fraud detection using computational intelligence,” Expert Systems with Applications, vol. 35, no. 4, pp. 1721–1732, 2008.

[6] S. Bhattacharyya, S. Jha, K. Tharakunnel, and J. C., “Data mining for credit card fraud: A comparative study,” Decision Support Systems, vol. 50, no. 3, pp. 602–613, 2011. [

7] N. S. Halvaiee and M. K. Akbari, “A novel model for credit card fraud detection using Artificial Immune Systems,” Applied Soft Computing, vol. 24, pp. 40–49, 2014.

[8] S. Panigrahi, A. Kundu, S. Sural, and A. K. Majumdar, “Credit card fraud detection: A fusion approach using Dempster–Shafer theory and Bayesian learning,” Information Fusion, vol. 10, no. 4, pp. 354–363, 2009. [9] N. Mahmoudi and E. Duman, “Detecting credit card fraud by modified Fisher discriminant analysis,” Expert Systems with Applications, vol. 42, no. 5, pp. 2510–2516, 2015.

[10] D. Sánchez, M. A. Vila, L. Cerda, and J. M. Serrano, “Association rules applied to credit card fraud detection,” Expert Systems with Applications, vol. 36, no. 2, pp. 3630–3640, 2009.

[11] E. Duman and M. H. Ozcelik, “Detecting credit card fraud by genetic algorithm and scatter search,” Expert Systems with Applications, vol. 38, no. 10, pp. 13057–13063, 2011.

[12] P. Ravisankar, V. Ravi, G. R. Rao, and I. Bose, “Detection of financial statement fraud and feature selection using data mining

60techniques,” Decision Support Systems, vol. 50, no. 2, pp. 491–500, 2011.

[13] E. Kirkos, C. Spathis, and Y. Manolopoulos, “Data mining techniques for the detection of fraudulent financial statements,” Expert Systems with Applications, vol. 32, no. 4, pp. 995–1003, 2007.

[14] F. H. Glancy and S. B. Yadav, “A computational model for financial reporting fraud detection,” Decision Support Systems, vol. 50, no. 3, pp. 595–601, 2011.

[15] D. Olszewski, “Fraud detection using self-organizing map visualizing the user profiles,” Knowledge-Based Systems, vol. 70, pp. 324–334, 2014.

[16] J. T. Quah and M. Sriganesh, “Real-time credit card fraud detection using computational intelligence,” Expert Systems with Applications, vol. 35, no. 4, pp. 1721–1732, 2008.

[17] E. Rahimikia, S. Mohammadi, T. Rahmani, and M. Ghazanfari, “Detecting corporate tax evasion using a hybrid intelligent system: A case study of Iran,” International Journal of Accounting Information Systems, vol. 25, pp. 1–17, 2017.

[18] I. T. Christou, M. Bakopoulos, T. Dimitriou, E. Amolochitis, S. Tsekeridou, and C. Dimitriadis, “Detecting fraud in online games of chance and lotteries,” Expert Systems with Applications, vol. 38, no. 10, pp. 13158–13169, 2011.

[19] C. F. Tsai, “Combining cluster analysis with classifier ensembles to predict financial distress” Information Fusion, vol. 16, pp. 46–58, 2014.

[20] F. H. Chen, D. J. Chi, and J. Y. Zhu, “Application of Random Forest, Rough Set Theory, Decision Tree and Neural Network to Detect Financial Statement Fraud–Taking Corporate Governance into Consideration,” In International Conference on Intelligent Computing, pp. 221–234, Springer, 2014.

[21] Y. Li, C. Yan, W. Liu, and M. Li, “A principle component analysisbased random forest with the potential nearest neighbor method for automobile insurance fraud identification,” Applied Soft Computing, to be published. DOI: 10.1016/j.asoc.2017.07.027.

[22] S. Subudhi and S. Panigrahi, “Use of optimized Fuzzy C-Means clustering and supervised classifiers for automobile insurance fraud detection,” Journal of King Saud University-Computer and Information Sciences, to be published. DOI: 10.1016/j.jksuci.2017.09.010.

[23] M. Seera, C. P. Lim, K. S. Tan, and W. S. Liew, “Classification of transcranial Doppler signals using individual and ensemble recurrent neural networks,” Neurocomputing, vol. 249, pp. 337-344, 2017.

[24] E. Duman, A. Buyukkaya, and I. Elikucuk, “A novel and successful credit card fraud detection system Implemented in a Turkish Bank,” In IEEE 13th International Conference on Data Mining Workshops (ICDMW), pp. 162–171, 2013.

[25] C. Phua, K. Smith-Miles, V. Lee, and R. Gayler, “Resilient identity crime detection,” IEEE Transactions on Knowledge and Data Engineering, vol. 24, no. 3, pp. 533–546, 2012.

[26] M. W. Powers, “Evaluation: From Precision, Recall and F-Measure to ROC, Informedness, Markedness & Correlation” Journal of Machine Learning Technologies, vol. 2, no. 1, pp. 37–63, 2011.

[27] Credit Card Fraud Detection [Online]. Available: https://www.kaggle.com/dalpozz/creditcardfraud

[28] R. Saia and S. Carta, “Evaluating Credit Card Transactions in the Frequency Domain for a Proactive Fraud Detection Approach,” In Proceedings of the 14th International Joint Conference on eBusiness and Telecommunications, vol. 4, pp. 335–342, 2017.

[29] ISO 8583-1:2003 Financial transaction card originated messages [Online]. Available: https://www.iso.org/standard/31628.html